

MATERIAL DE APOIO

LAYOUT FLEXBOX

O Flexbox tem como meta ser um modo mais eficiente para criar layouts, alinhar e distribuir espaços entre itens em um container, mesmo quando as dimensões destes itens são desconhecidas e/ou dinâmicas (daí o termo "**flex**").

O objetivo deste layout é dar ao contêiner a capacidade de alterar a largura ou altura (e ordem) de seus itens para melhor preencher o espaço (principalmente para acomodar todos os tipos de dispositivos de exibição e tamanhos de tela).

Um contêiner flexível expande os itens para preencher o espaço livre disponível ou os reduz para evitar o transbordamento.

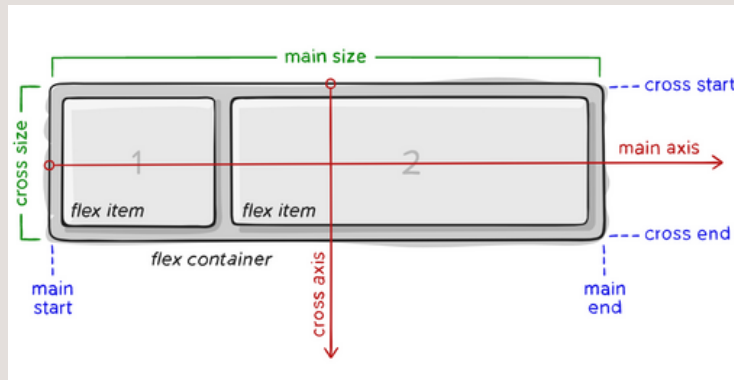
Mais importante ainda, o layout do flexbox é independente de direção, em oposição aos layouts regulares.

CONCEITOS BÁSICOS SOBRE FLEXBOX

Como o flexbox é um módulo e não uma única propriedade, ele envolve muitas coisas, incluindo todo o seu conjunto de propriedades. Alguns deles devem ser definidos no contêiner (elemento pai, conhecido como "flex container"), enquanto os outros devem ser definidos nos filhos (chamados de "flex items").

Se o layout "regular" for baseado em direções de fluxo em bloco e em linha, o layout flexível será baseado em "flex-flow directions" (do inglês direções de fluxo flexível).

Confira a figura abaixo que traz alguns detalhes sobre layout flexível.



Os itens serão dispostos seguindo o eixo principal (main axis) ou o eixo cruzado (cross axis).

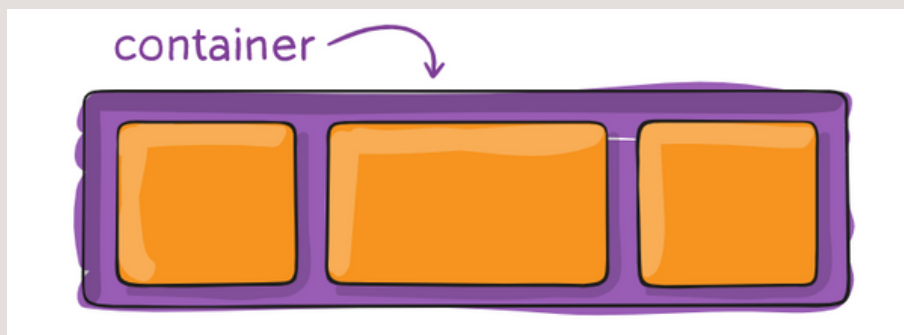
- **main axis** - Eixo principal de um contêiner flexível é o eixo principal ao longo do qual os itens flexíveis são dispostos. Cuidado, não é necessariamente horizontal; depende da propriedade flex-direction.
- **main-start | main-end** - Os flex items são colocados dentro do contêiner começando do main-start e indo para o main-end.
- **main size** - A largura ou altura de um item flexível, o que estiver na dimensão principal, é o tamanho principal do item. A propriedade de tamanho principal do item flexível é a propriedade largura ou altura, o que estiver na dimensão principal.
- **cross axis** - O eixo perpendicular ao eixo principal é chamado de eixo cruzado. Sua direção depende da direção do eixo principal.
- **cross-start | cross-end** - As linhas flexíveis são preenchidas com itens e colocadas no contêiner, começando no lado da ponta cruzada do contêiner flexível e indo em direção ao lado da ponta cruzada.
- **cross size** - A largura ou altura de um item flexível, o que quer que esteja na dimensão cruzada, é o tamanho cruzado do item. A propriedade de tamanho cruzado é qualquer largura ou altura que esteja na dimensão cruzada.



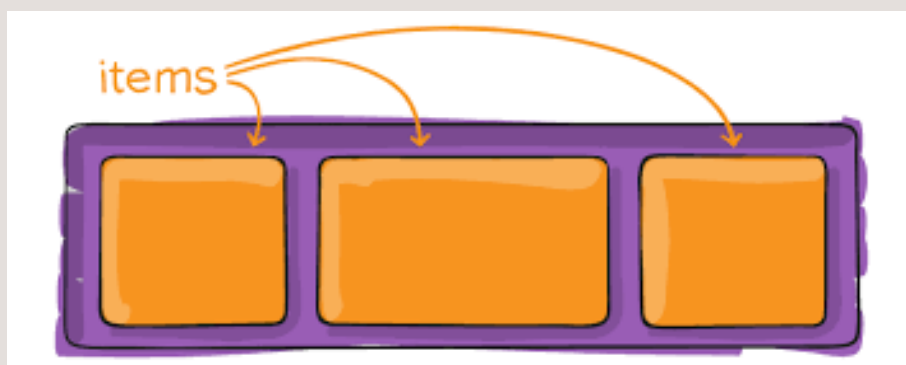
Propriedades Flexbox

Antes de explorar as propriedades, vamos conferir a nomenclatura padrão:

1. **Container** é um elemento que contém outros elementos. O container é considerado o elemento **Parent** (pai ou pais).



2. **Children** são os itens internos (**filhos**) dentro de um container, ou conforme a analogia dos próprios nomes, são os filhos de elemento pai.

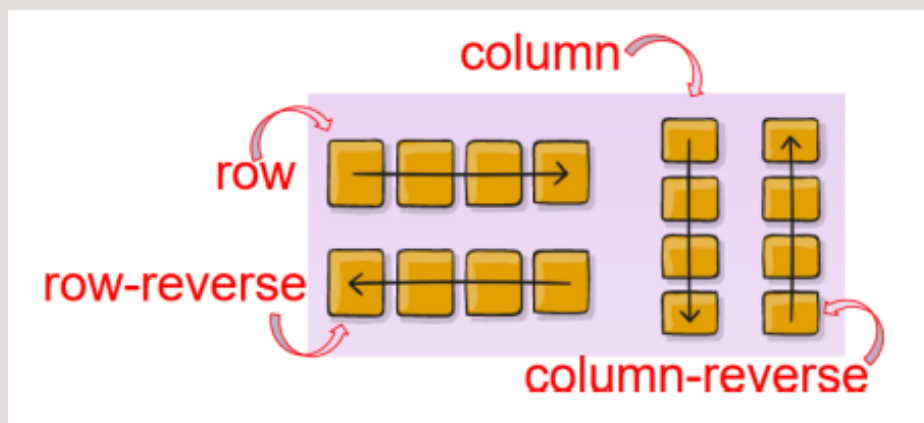


PROPRIEDADES DO PARENT

- **display** - Define um flex container. Pode ser flex ou inline-flex, dependendo do valor fornecido. Ele permite um contexto flexível para todos os seus filhos diretos, isto é tudo que estiver dentro do container. Confira abaixo um exemplo desta propriedade definida como flex.

```
1 .container {  
2   display: flex; /* or inline-flex */  
3 }
```

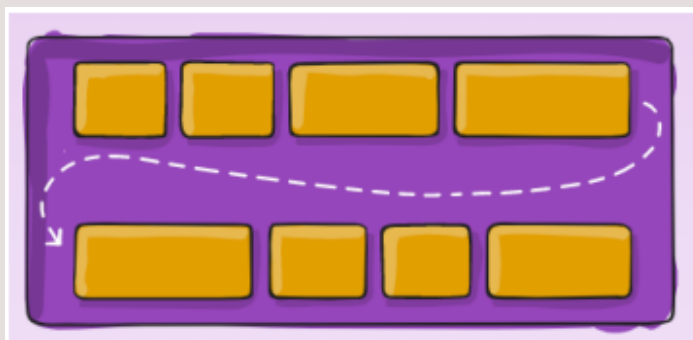
- **flex-direction** - Estabelece o eixo principal, definindo assim a direção que os flex items são colocados no flex container. Flexbox é (além da embalagem opcional) um conceito de layout de direção única. Pense nos itens flexíveis principalmente em linhas horizontais ou colunas verticais, como mostra a figura abaixo.



Os possíveis valores para esta propriedade são: `row`, `row-reverse`, `column` ou `column-reverse`. Confira abaixo um exemplo da propriedade `flex-direction` definida como `row`.

```
1 .container {  
2   flex-direction: row /* or row-reverse or column or column-reverse */;  
3 }
```

- **flex-wrap** = Por padrão, os itens flexíveis tentarão todos se ajustar em uma linha. Você pode alterar isso e permitir que os itens sejam embalados conforme necessário. Veja uma figura ilustrativa deste comportamento.



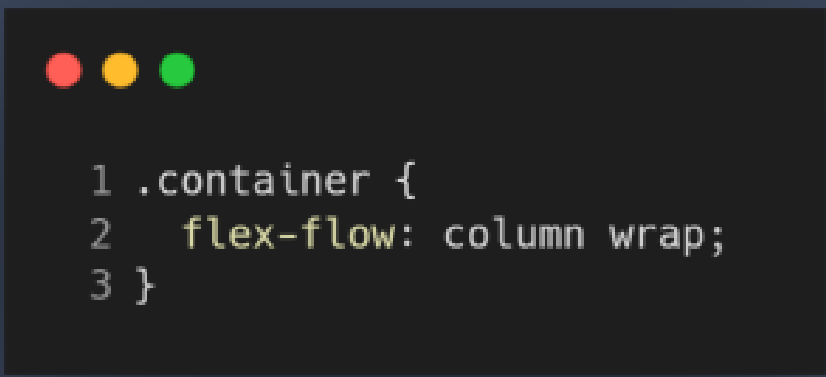
Para definir a quebra de linha podemos usar as seguintes opções:

- **nowrap (padrão)**: todos os flex items estarão em uma única linha.
- **wrap**: os itens flexíveis serão quebrados em várias linhas, de cima para baixo.
- **wrap-reverse**: os itens flexíveis serão quebrados em várias linhas de baixo para cima. Confira abaixo um exemplo da propriedade flex-wrap definida como wrap-reverse.

```
1 .container {  
2   flex-wrap: wrap-reverse /* nowrap or wrap */;  
3 }
```

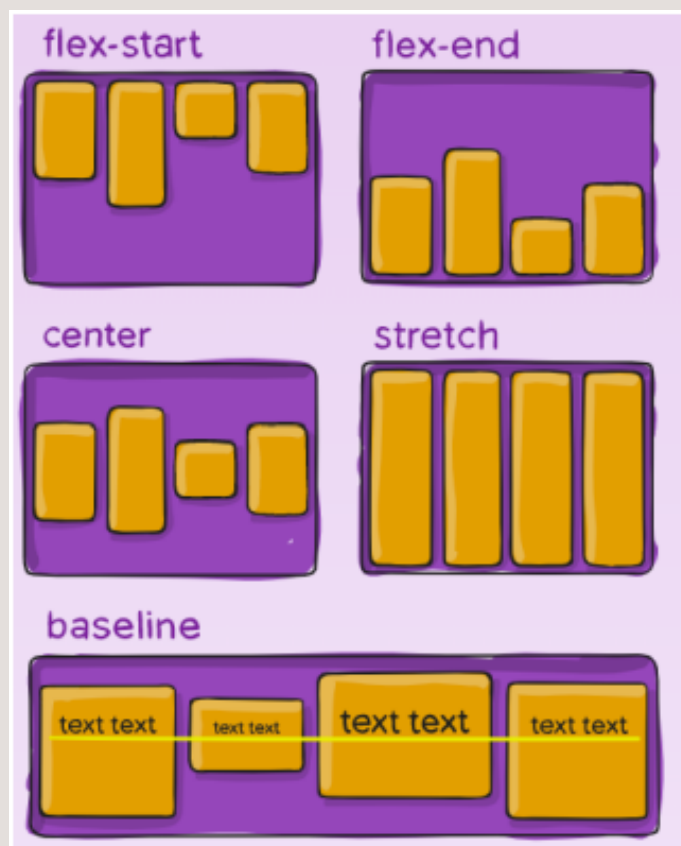
Para mais detalhes, visite os exemplos da propriedade flex-wrap, no link: <https://css-tricks.com/almanac/properties/f/flex-wrap/>

- **flex-flow** - É uma abreviação para as propriedades flex-direction e flex-wrap, que juntas definem os eixos principal e transversal do flex container. O valor padrão é row nowrap. Os possíveis valores são as combinações entre [row, row-reverse, column ou column-reverse] e [nowrap wrap wrap-reverse]. Confira abaixo um exemplo da propriedade flex-flow definida como a combinação [column] e [wrap].



```
1 .container {  
2   flex-flow: column wrap;  
3 }
```

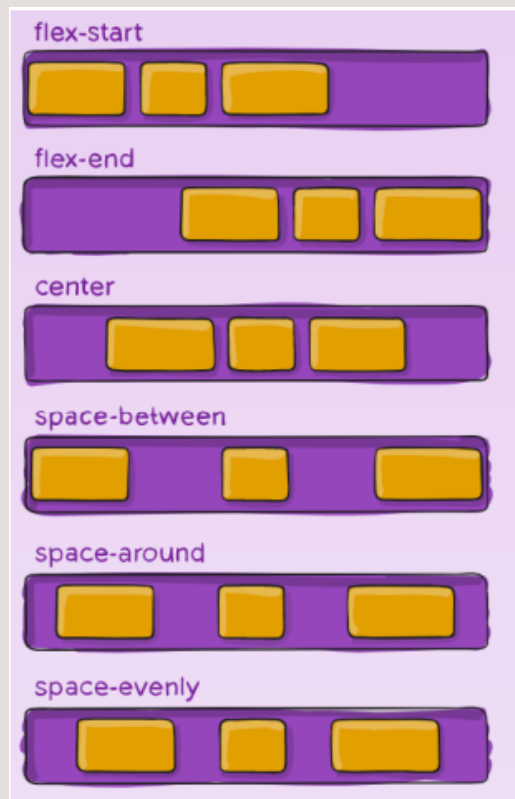
- **align-items** - Define o comportamento padrão de como os itens flexíveis são dispostos ao longo do eixo cruzado na linha atual. Pense nisso como a versão do conteúdo justificado para o eixo cruzado (perpendicular ao eixo principal). Veja abaixo exemplos para a propriedade align-items.



Confira abaixo um exemplo da propriedade `align-items` definida como `flex-start`.

```
1 .container {  
2   align-items: flex-start  
3 }
```

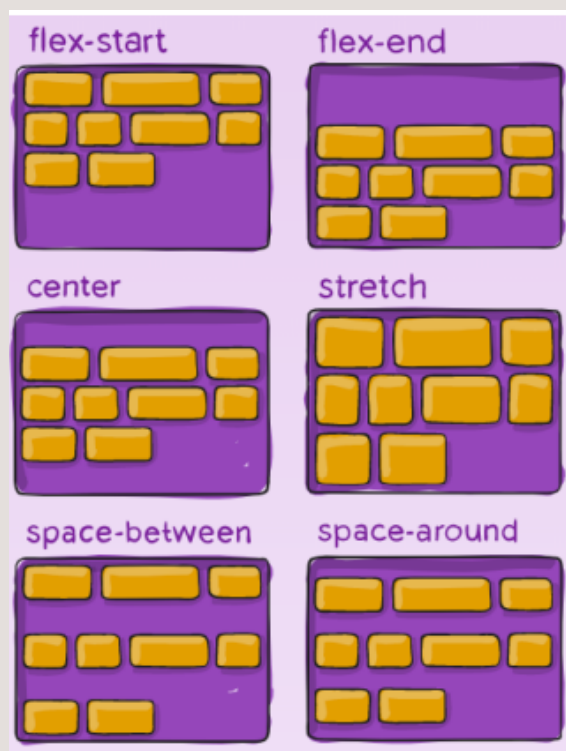
- **justify-content** - Define o alinhamento ao longo do eixo principal. Ajuda a distribuir o espaço livre extra restante quando todos os itens flexíveis em uma linha são inflexíveis ou são flexíveis, mas atingiram seu tamanho máximo. Ele também exerce algum controle sobre o alinhamento dos itens quando eles ultrapassam a linha. Veja na figura ao lado as possíveis opções para esta propriedade.



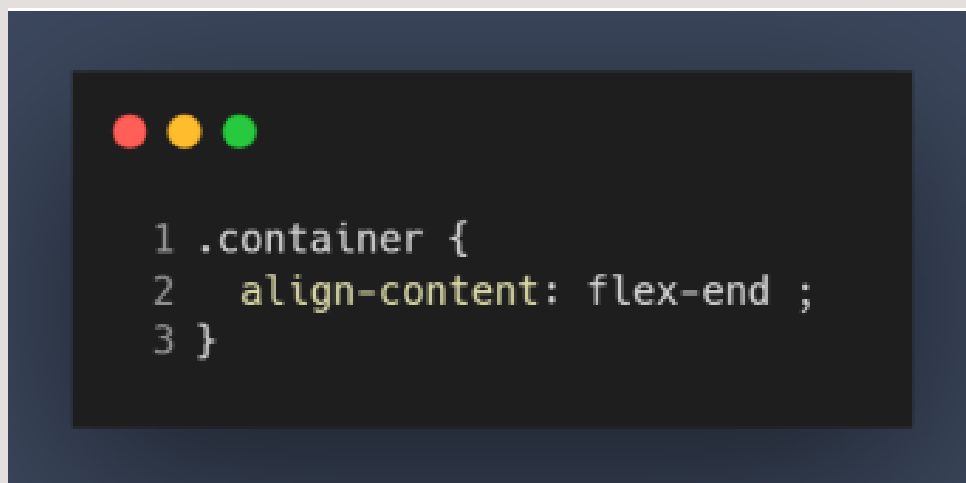
Confira a seguir um exemplo da propriedade `justify-content` definida como `flex-start`. Existem também duas palavras-chave adicionais que você pode associar a esses valores: `safe` e `unsafe`. Usar o `safe` garante que, independentemente de como você faça esse tipo de posicionamento, você não pode empurrar um elemento para que ele fique fora da tela (por exemplo, fora do topo) de forma que o conteúdo também não possa ser rolado (chamado de "perda de dados").

```
1 .container {  
2   justify-content: flex-start;  
3 }
```

- **align-content** – Alinha as linhas de um flex container quando há espaço extra no eixo cruzado, semelhante quando o justify-content alinha itens individuais dentro do eixo principal. Veja abaixo exemplos desse alinhamento.

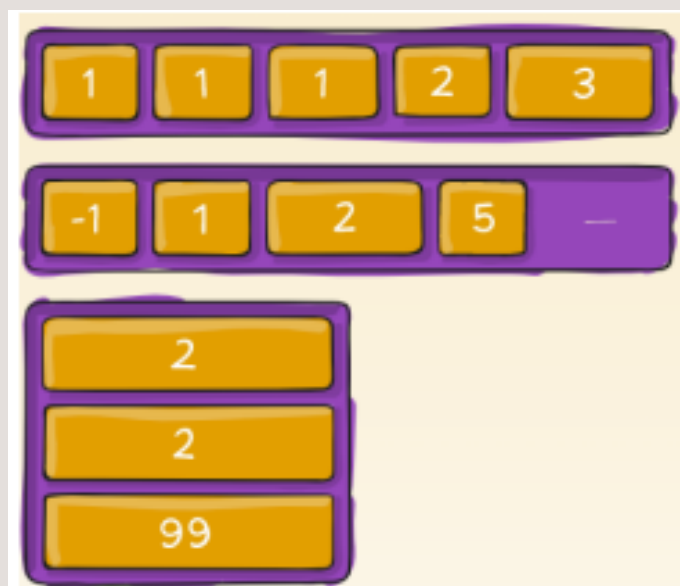


Observação: esta propriedade só tem efeito em contêineres flexíveis de várias linhas, onde flex-flow é definido como wrap ou wrap-reverse). Um contêiner flexível de linha única (ou seja, onde flex-flow é definido com seu valor padrão, no-wrap) não refletirá o conteúdo de alinhamento. Confira abaixo um exemplo da propriedade align-content definida como flex-end.



PROPRIEDADES DO CHILDREN

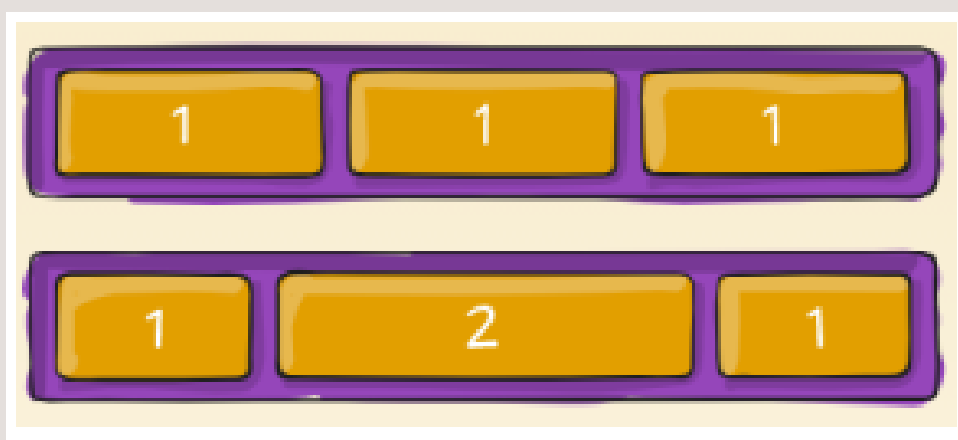
- **order** – Por padrão, os itens flexíveis são dispostos na ordem de origem. No entanto, a propriedade order controla a ordem em que aparecem no flex container. Os valores devem ser numéricos e podem receber valores negativos e positivos, confira na figura abaixo alguns exemplos de order.



Confira abaixo um exemplo da propriedade order definida como 5:

```
1 .item {  
2   order: 5; /* default is 0 */  
3 }
```

flex-grow - Define a capacidade de um item flexível crescer, se necessário. Aceita um valor sem unidade que serve de proporção. Ele determina a quantidade de espaço disponível dentro do flex container que o item deve ocupar. Confira na figura abaixo alguns exemplos de flex-grow.



Se todos os itens tiverem flex-grow definido como 1, o espaço restante no contêiner será distribuído igualmente para todos os filhos. Se um dos filhos tiver o valor 2, o espaço restante ocupará o dobro do espaço dos outros (ou tentará, pelo menos). Confira abaixo um exemplo da propriedade flex-grow definida como 2:

```
1 .item {  
2   flex-grow: 2; /* default 0 */  
3 }
```

- **flex-shrink** - Define a capacidade de um item flexível encolher, se necessário. No exemplo abaixo o valor foi configurado em 3, isso significa que o item terá a capacidade de encolher 3 vezes mais do que os demais elementos.

```
1 .item {  
2   flex-shrink: 3; /* default 1 */  
3 }
```

- **flex-basis** - Define o tamanho padrão de um elemento antes que o espaço restante seja distribuído. Pode ser um comprimento (por exemplo, 20%, 200px, 5 rem, etc.) ou uma palavras-chave como por exemplo auto, max-content, min-content, fill e fit-content. Confira abaixo um exemplo desta propriedade definida com fill.

```
1 .item {  
2   flex-basis: fill; /* default auto */  
3 }
```

- **flex** - Esta é a abreviação de flex-grow, flex-shrink e flex-basis combinados. O segundo e terceiro parâmetros (flex-shrink e flex-basis) são opcionais. Confira abaixo um exemplo desta propriedade definida com flex-grow=2, flex-shrink=3 e flex-basis=10%

```
1 .item {  
2   flex: 2 3 10%  
3 }
```

- **align-self** - Permite que o alinhamento padrão (ou aquele especificado por align-items) seja substituído por itens flexíveis individuais. Os possíveis valores para esta propriedade são: auto | flex-start, flex-end, center, baseline ou stretch. Confira abaixo um exemplo desta propriedade definida com flex-start.



```
1 .item {  
2   align-self: flex-start;  
3 }
```

REFERÊNCIAS

1. [A Complete Guide to Flexbox](#)

BONS ESTUDOS!

